

1) HTML5 Geolocation (built-in)

What you get: User's current {lat, lon} one-time or continuous.

How to use: call `navigator.geolocation.getCurrentPosition` on login or profile edit → store in your state → POST to PHP.

PHP: add `users(lat DECIMAL(9,6), lon DECIMAL(9,6), last_located_at)` and a `/api/update_location.php` that writes coordinates for the logged-in user.

Good for: “Find dogs within 10 miles,” “Events near me,” auto-centering the map.

Gotchas: Must be on HTTPS (or `http://localhost` during dev). User can deny permission; handle gracefully.

2) Leaflet + OpenStreetMap tiles (map UI)

What you get: Interactive map (pan/zoom), markers, popups, clustering, drawing.

How to use: add `leaflet + react-leaflet` → render a `<MapContainer center={{lat,lon}}>` and drop markers for dogs/events pulled from PHP (e.g., `/api/nearby_dogs.php?lat=...&lon=...&radius=...`).

CSS: import Leaflet CSS once.

Good for: Visual discovery, “hosted events” pins, selecting a meetup spot.

Gotchas: Follow OSM attribution rules; don't hammer the free tile server.

3) Turf.js (client-side matching logic)

What you get: Distance, buffering, “within radius,” sorting by proximity—entirely in the browser.

How to use: Fetch a list of candidate dogs/events from PHP (with their {lat,lon}), then in React run `turf.distance([lonA,latA],[lonB,latB], {units: 'miles'})` to:

- filter: `<= radius`
- sort: nearest first
- group: cluster parks/playdates

Good for: Fast “near me” results without extra server work on small/medium lists.

Gotchas: If your list grows large (thousands), move the radius filter to PHP/SQL for performance (see #4).

4) PHP + MySQL/Postgres geospatial (server-side matching)

What you get: Efficient radius queries on the server (scales better).

Option A simple (Haversine in SQL):

Add a computed distance in your **SELECT** using the Haversine formula and a **WHERE distance <= :radius**. **Index (lat, lon)** to keep it snappy.

Option B proper geo:

- **MySQL:** use **SPATIAL INDEX** on **POINT(lon,lat)** (SRID 4326) and **ST_Distance_Sphere/ST_DWithin** style queries.
- **Postgres + PostGIS:** best-in-class geo types, **geography** + **ST_DWithin** for precise meters and easy indexing.

Good for: “Show 50 closest dogs,” “events within 15 km,” pagination.

Gotchas: Store lon/lat consistently; validate inputs; clamp radius.

Suggest:

- Use HTML5 Geolocation to get user coordinates
- Use Leaflet + OpenStreetMap to show maps
- Use Turf.js to filter and rank nearby dogs/events